INF1009

< Lab Group P3-6 >

Abstract Engine

Student Name	Student ID
Abburu Sai Tanvi	2503638
Chourasia Anushka	2503769
Chern De Qian	2502760
Ang Wei Long	2500447
Adora Goh Shao Qi	2502513
Goh Jason Fadil	2502643

1. Introduction ---------- 3
1.1. Overview of Abstract Engine ---------- 3
1.2. Key Features Implemented ---------- 3
1.3. Unique Features/Enhancements ---------- 4
1.4. High-Level summary of systems/concepts ---------- 4
2. System Design and Implementation ---------- 4
2.1. Architectural Overview ---------- 4
3.2 Managers and Their Responsibilities ---------- 5
3.2.1 CollisionManager ---------- 5
3.2.2 TimeManager ---------- 7
3.2.3 SceneManager ---------- 8
3.2.4 EntityManager ---------- 9
3.2.5 Input/Output Manager ---------- 10
3.2.6 MovementManager ---------- 11
3.3 Application of OOP Principles ---------- 12
3.3.1 CollisionManager ---------- 12
3.3.2 TimeManager ---------- 13
3.3.3 Entity Manager ---------- 13
3.3.4 SceneManager ---------- 14
3.3.5 Input/Output Manager ---------- 14
3.3.6 MovementManager ---------- 15
3. UML Diagram ---------- 16
4. Team Contribution ---------- 16
5. Appendix A: Program Structure ---------- 18
6. Appendix B: TeamAId Usage ---------- 19

1. Introduction

1.1. Overview of Abstract Engine

For Part 1 of this project, our team developed an Abstract Engine. The purpose of this engine is to provide a general and reusable foundation for building different types of simulations. Instead of focusing on a specific game or scenario, the engine concentrates on implementing the core systems that are commonly required in most simulation environments.

The engine is designed to manage the core functionalities, which include handling, entity updates, movement processing, collision detection, and input/output management. By separating these systems from context-specific logic, the engine becomes more modular and adaptable

Overall, the Abstract Engine acts as the backbone of the system, ensuring scalability, maintainability, and proper application of object-oriented programming principles.

1.2. Key Features Implemented

The engine follows a modular architecture consisting of the following main components:

Game Master

The Game Master acts as the central coordinator of the engine. It initializes all managers and has the main control for the game. It ensures that update calls occur in the correct order and all subsystems operate smoothly.

Scene Manager

The Scene Manager controls which scene is currently active. It handles the transitions and ensures that only one scene is running at any given time.

Entity Manager

The Entity Manager stores and updates all entities within the active scene. It ensures that the objects behave correctly during each frame

Collision Manager

The Collision Manager handles the detection and handling of interactions between entities. It retrieves active entities from the Entity Manager and does a collision check to determine when and where a collision occurs. When it detects a collision, it executes the proper response logic depending on the entity. There is separate detection and handling logic.

Input/Output Manager

This manager handles the user input and output operations

Time Manager

The Time Manager calculates the delta time to ensure consistent behaviour across different frame rates.

Movement Manager

The Movement Manager is responsible for updating entity movement in a consistent and frame-independent manner. It applies movement using each entity's speed and the delta time provided by the Time Manager, ensuring that gameplay feels consistent across different frame rates. Movement updates modify an entity's

position (and optionally velocity/state), after which the updated positions can be used by the Collision Manager to check for interactions. This keeps movement logic centralized and reduces duplicated movement code across entity classes

1.3. Unique Features/Enhancements

Our Abstract Engine includes several innovative features that enhance functionality and demonstrate advanced design principles. The SceneManager implements a dual transition mechanism with both immediate (startScene) and queued (requestNextScene/loadNext) modes for responsive UI while ensuring frame-boundary safety. Scene storage uses a Map-based architecture for O(1) lookup rather than linear search, enabling dynamic runtime registration. The collision system features layer-based filtering allowing selective collision detection between entity types (player-enemy, player-wall) without complex conditionals. Our action-based input abstraction decouples game logic from physical keys through InputAction enumeration, supporting multiple input devices seamlessly. The entity system supports runtime component addition through a flexible component map, demonstrating composition over inheritance.

1.4. High-Level summary of systems/concepts

The Abstract Engine implements a manager-based architecture where GameMaster orchestrates five specialized managers handling distinct responsibilities: SceneManager (state control), EntityManager (object lifecycle), CollisionManager (physics events), MovementManager (position updates), and InputOutputManager (user input). The design follows the State Pattern with scenes representing discrete game phases, event-driven collision response through listener interfaces decoupling detection from response, and delta-time physics ensuring frame-rate independent behavior. All managers operate on abstract types (Scene, Entity, InputProcessor) enabling polymorphism—GameMaster's loop works with any scene or entity type without knowing implementations. The architecture prioritizes extensibility and reusability over game-specific optimization: adding new scenes, entities, or behaviors requires zero modifications to existing manager code, demonstrating the Open-Closed Principle. Each component follows defined lifecycle management (initialization → active → cleanup) and adheres to the Single Responsibility Principle, resulting in zero context-specific code, complete decoupling through abstract interfaces, and an engine that is both simple to understand yet powerful enough to support complex interactive applications with professional code quality.

2. System Design and Implementation

2.1. Architectural Overview

Our engine follows a class-based architecture enhanced with lightweight component ideas. We selected this approach for clarity, readability, and alignment with OOP learning objectives. The architecture centers around the GameMaster, which maintains references to all managers. (see Appendix A) The high-level execution flow is as follows:

1. Game Master initializes managers.
2. Scenes are registered with Scene Manager.

3. The initial scene is activated.
4. The main loop begins execution.
5. Each frame:
    - Time Manager calculates delta time.
    - Scene Manager updates the active scene.
    - Entity Manager updates all entities.
    - Movement Manager processes entity movement (integrated within entity updates).
    - Collision Manager checks for collisions.
    - Scene Manager renders the scene.
6. This structured loop ensures deterministic and consistent behaviour.

The architecture emphasizes:

- Decoupling of systems
- Clear data flow
- Minimal interdependencies
- Scalability for future extension

3.2 Managers and Their Responsibilities

3.2.1 CollisionManager

Purpose

The collision manager is responsible for detecting and responding to collisions between entities in every frame of execution. In every cycle update, the manager retrieves the list of active entities from the EntityManager and checks for collisions. Once a collision has been detected, the manager calls the corresponding collision response function on the colliding entities.

Core Collision components:

- ICollisionListener - Interface defining the callback method that gets triggered when two entities collide.
- DetectCollision - Stores layer-pair rules that determine which entity types should be checked for collisions.
- HandleCollision - Stores and executes the response handlers registered for specific layer combinations.

Attributes

Attribute	Type	Description
entityManager	EntityManager	Holds a reference to the entity manager used to get the list of active entities for collision checking
listeners	List<ICollisionListener>	collision listeners that are notified whenever any collision happens.

handleCollision	HandleCollision	manages and executes the actual per-layer collision handling logic
detectCollision	DetectCollision	stores collision rules and decides if two layers should be checked for collision

Methods

Method	Description
addListener(ICollisionListener listener)	Registers a global collision listener that gets notified about all collision events.
setEntityLayer(Entity entity, String layer)	Assign an entity to a specific collision layer, determining which collision rules apply
getEntityLayer(Entity entity)	Retrieves the collision layer name assigned to a given entity.
removeListener(ICollisionListener listener)	Unregisters a previously added global collision listener.
detectCollision(): void	Triggers collision detection for all active entities in the current frame.
handleCollision(Entity a, Entity b, String layerA, String layerB): void	Processes a detected collision by invoking the appropriate layer-specific handler and notifying all listeners.
registerCollision(String layerA, String layerB, ICollisionListener handler)	Registers a collision handler for a specific pair of layers, defining how collisions between those layers should be handled.
removeCollision(String layerA, String layerB): void	Removes a previously registered collision handler for a specific layer pair.
setLayerCollision(String layerA, String layerB, boolean detect): void	Configures whether two layers should detect collisions with each other, enabling fine-grained collision filtering.
addCollidable(Entity entity, String layer): void	Adds an entity to the collision system and assigns it to a specific layer.
removeCollidable(Entity entity): void	Removes an entity from the collision system, preventing it from participating in collision detection.

Integration with Other Systems

The Collision Manager integrates with the Abstract Engine's core systems to provide coordinated collision detection and response.

- **Entity Manager** - Provides the list of active entities to check for collisions each frame.
- **Movement Manager** - Runs before collision detection so entities are in their updated positions when checked.
- **Time Manager** - Handlers may use deltaTime for time-based effects like cooldowns.
- **Scene Manager** - Each scene can register its own collision rules and clear them on transition.
- **Input/Output Manager** - Collision events can trigger sounds through the Speaker.

3.2.2 TimeManager

Frame-Independent Updates

The raw delta time is sent to TimeManagers update method at the start of every frame, in GameMasters tick method. TimeManager then looks at the timescale and if the game is paused. It uses these to process the delta time and stores it in deltaTime. The other managers get this processed value using getDeltaTime. This helps them update things at the rate no matter the frame rate. For example MovementManager multiplies an entitys speed by deltaTime when it updates the entitys position. This keeps the game moving at the speed whether it runs at 30 FPS or 120 FPS.

Time Flow Control

TimeManager offers control over time via pause and time scale. When pause() is invoked, TimeManager sets deltaTime to 0, resulting in the freezing of all movements and animations as the game loop runs. Time scale enables effects, where setTimeScale(0.5f) slows down actions to half speed for "bullet time" effects and setTimeScale(2.0f) accelerates time for fast-forwarding.

Time Lifecycle

Every frame, time flows through the following stages:

- **Raw Capture** — GameMaster captures the raw frame time from the rendering system.
- **Processing** — TimeManager.update() applies timeScale and pause state to produce deltaTime.
- **Distribution** — All managers query getDeltaTime() for their updates.
- **Accumulation** — totalTime is incremented for long-term time tracking.

Attributes

Attribute	Type	Description
deltaTime	float	The scaled time elapsed since the last frame in seconds. Used by all systems for frame-independent updates. Becomes 0 when paused.
totalTime	float	The cumulative time since the game started in seconds. Continuously increases regardless of pause or time scale.
timeScale	float	Speed multiplier for game time (default 1.0). Values less than 1.0 create slow motion; values greater than 1.0 speed up time.
isPaused	boolean	Flag indicating whether game time is currently paused. When true, deltaTime becomes 0.

Methods

Method	Description
update(dt)	Updates internal time tracking using raw delta time. Calculates scaled deltaTime and increments totalTime. Must be called once per frame.
getDeltaTime()	Returns the current scaled delta time for frame-independent calculations. Returns 0 when paused.
getTotalTime()	Returns the total elapsed time since game initialization.
getTimeScale()	Returns the current time scale multiplier.

isPaused()	Returns whether the game time is currently paused.
setTimeScale(scale)	Sets the time scale multiplier, clamped between 0.0 and 10.0. Used for slow motion or speed-up effects.
pause()	Pauses game time. Subsequent getDeltaTime() calls return 0 until resume() is called.
resume()	Resumes game time. Time tracking continues normally.
togglePause()	Switches between paused and resumed states.
reset()	Resets all time tracking to initial state.

3.2.3 SceneManager

Scene Registration and Transitions

At engine startup, GameMaster creates the three concrete scenes and registers them with the SceneManager using addScene(). The engine then calls startScene("menu") to begin. When the user selects Start Game, GameMaster detects this through the active scene's state and calls startScene("main"), which automatically stops MenuScene and starts MainScene. The same mechanism transitions to EndScene when a game-over collision is detected.

Per-Frame Delegation

Inside GameMaster's main game loop, two calls are made each frame: sceneManager.update(deltaTime) and sceneManager.render(). SceneManager simply forwards these to currentScene, so the active scene handles all logic and drawing. No other scene is touched.

Scene Lifecycle

Every scene transitions through the following stages, enforced by SceneManager:

- Registration — addScene() stores the scene in the map with no initialisation.
- Activation — startScene() calls stop() on the old scene, then start() on the new one.
- Running — update(dt) and render() are forwarded to the active scene every frame.
- Deactivation — stop() is called when another scene is requested, freeing resources.

Purpose and Overview

The SceneManager is responsible for controlling the high-level flow of the application. It acts as a state machine that determines which scene is currently active and handles transitions between scenes. A scene represents one self-contained state of the game — such as the main menu, active gameplay, or the end screen. The SceneManager ensures exactly one scene is active at any time, delegating per-frame update and render calls to that scene while keeping all others idle.

Attributes

Attribute	Type	Description
scenes	Map<String, Scene>	Stores all registered scenes keyed by name (e.g. 'menu', 'main', 'end'). Allows fast lookup by name.
currentScene	Scene	Reference to the currently active scene. Only this scene receives update() and render() calls each frame.

Methods

Method	Description
addScene(name, scene)	Registers a scene under a string key for later retrieval and activation.
startScene(name)	Stops the current scene, then starts the named scene — the primary transition method.
stopScene(name)	Stops a named scene and clears the current scene reference if it was active.
getActiveScene()	Returns the current running scene so external code can inspect its state.
requestNextScene(name)	Queues a scene transition to be executed on the next loadNext() call.
loadNext()	Applies the queued transition — stops current and starts the pending scene.
update(deltaTime)	Delegates the per-frame update call to the currently active scene.
render()	Delegates the per-frame render call to the currently active scene.

3.2.4 EntityManager

Purpose: Centralizes the management of all game entities, handling their entire lifecycle including creation, updates, rendering, and removal to prevent concurrent modification errors during the game loop.

Objects Managed: Entity objects (stored in Array List collections).

Logic Flow Rationale: To ensure stability in the engine, the entity manager employs a **buffer system**. Instead of modifying the main entity list while the game loop is mid-iteration which causes crashes, new entities are queued in entitiesToAdd and removed via entitiesToRemove. These changes are synchronized at the start of every frame by the processPendingChanges() method.

High-Level Methods Only:

Method	Description
addEntity(Entity entity):	Registers a new entity for management by adding it to a pending list.
removeEntity(Entity entity)	Marks an entity for removal and triggers its disposal.
updateAll(float deltaTime)	Iterates through active entities to trigger their individual logic updates.
renderAll()	Coordinates the rendering process for all currently active entities
getEntitiesByType(Class<T> type)	Retrieves a filtered list of entities belonging to a specific subclass.

3.2.5 Input/Output Manager

Purpose: The input/output manager is responsible for managing all user interaction. It provides a unified interface for reading keyboard and mouse input, as well as handling display output. This manager is designed to be fully abstract, ensuring that the engine remains reusable and free from context-specific logic.

To support both simulation and real execution, the IO manager uses a dual-layer architecture, with abstract layer and LibGDX Integration Layer. This approach allows the engine to remain clean and modular while still integrating with LibGDX when needed for the actual game loop.

Responsibilities: Input Polling, in simulation mode, this resets per-frame states and allows controlled testing of input behaviour. In LibGDX mode, polling ensures that the engines receives updated hardware input from the LibGDX input system. This ensures consistent behaviour across both simulated and real environments.

Action-based input mapping maintains a mapping between high-level enums and actual key codes. This allows the engine to interpret input based on intent rather than raw key values. Bindings can be changed dynamically at runtime. This updates both the internal binding table and the keyboard's own action-binding map, ensuring that input changes consistently.

Display management through the display interface. This allows scenes and entities to perform operations such as clearing the screen, rendering output, and retrieving display dimensions without depending on a specific rendering framework.

Attributes

Attribute	Type	Description
Keyboard	Keyboard	Reference to the active keyboard device
Mouse	Mouse	Reference to the active mouse device
Display	Display	Reference to the display system used for rendering output
Bindings	Map<InputAction, Integer>	Stores the mapping between high-level input actions and key codes.

Methods

Method	Description
pollInput()	Polls the keyboard and mouse each frame to update their internal states.
getKeyboard()	Returns the active keyboard device.
getMouse()	Returns the active mouse device.
getDisplay()	Returns the active display implementation.
bind(InputAction action, int keyCode)	Rebinds an input action to a new key and updates the keyboard's binding table.

isPressed(InputAction action)	Checks whether the key mapped to the given action is currently pressed.
setBindings(List<InputBinding> inputBindings)	Applies a batch of input bindings using InputBinding objects.

3.2.6 MovementManager

Purpose and Overview

The Movement Manager is responsible for handling all entity movement logic within the Abstract Engine. Instead of Implementing a centralized manager class, the movement system is distributed across entity implementations through the IMovable Interface and movement capabilities built into the Entity hierarchy. This design allows each entity type to define its own movement behaviour while maintaining a consistent movement API.

The system integrates closely with the Entity Manager for state updates, the Collision Manager for movement validation, and the Time Manager for frame-rate independence through delta-time-based calculations.

Core Movement Components

The movement system consists of:

- IMovable Interface - Defines the movement contract for movable entities
- Entity Base Class - Provides position, velocity, and speed management
- Player Movement - User-controlled directional movement (200 pixels/second)
- NPC Movement - AI-driven autonomous behaviors (patrol, wander, idle, follow)

Attributes
Movement state is distributed across Entity Instances:

Attribute	**Type**	**Description**
Position	Vector2	Current entity position in 2D space (x, y coordinates)
Velocity	Vector2	Movement velocity vector determining direction and speed
Speed	float	Base movement speed multiplier in pixels per second
State	string	Current entity state ("idle", "moving", "patrolling", "wandering")

For NPC Entities specifically:

Attribute	Type	Description
behaviourType	string	NPC AI behavior pattern ("patrol", "wander", "idle", "follow")
patrolRadius	float	Maximum distance NPC can move from initial position
patrolTimer	float	Time counter for behavior timing and pattern calculation

Methods

Attribute	Description

move(dx, dy, deltaTime)	Applies directional input scaled by speed and delta time. Updates position and transitions state between "idle" and "moving"
update(deltaTime)	Per-frame update applying velocity to position. For NPCs, executes behavior-specific logic (patrol, wander)
resetPosition()	Returns entity to initial position, clears velocity, resets state to "idle"
setPosition(x, y)	Updates coordinates and synchronizes position vector
outOfBounds(minX, minY, maxX, maxY)	Boundary checking utility for movement validation

Integration with Other Systems

The Movement Manager integrates with the Abstract Engine's core system to provide coordinated entity motion

- Entity Manager - Maintains the list of movable entities and calls entity:update(deltaTime) each frame, triggering movement logic automatically
- Collision Manager - Uses updated entity positions (x, y, width, height, radius) for collision detection and can modify velocity or position in response to collisions.
- Time Manager - Provides deltaTime for frame-rate independant movement calculatinos, ensuring entitites move consistently at 200 pixels/second regardless of whether the game runs at 30 FPS or 120 FPS
- Scene Manager - Each scene maintains its own entities with independant movement states, using resetPosition() during scene transitions

3.3 Application of OOP Principles

3.3.1 CollisionManager

Principle	**How It Is Applied in CollisionManager**
Encapsulation	Collision data structures internally (entityManager, collisionHandlers, listeners, handleCollision, detectCollision, entityLayers) are kept private. External code cannot access or modify these directly. Interaction can only happen through public methods such as addCollidable, setEntityLayer, detectCollision, registerCollision, and setLayerCollision, ensuring controlled access to collision configuration.
Single Responsibility	The class is responsible only for collision management. It retrieves active entities, determines which pairs should be checked, performs collision detection, and dispatches events to handlers. It does not handle any other functions from other managers
Singleton Pattern	CollisionManager is instantiated once per game and reused throughout the application. Though it does not use a static getInstance() method, this single instance approach ensures all collision rules, layer configurations, and handler registrations are centralized, preventing conflicting collision states.
Abstraction	Entities are identified by abstract string layers rather than concrete types. Games define their own layer names and provide collision behaviour through

	ICollisionListener implementations. The collision detection algorithm is hidden in private methods, exposing only high-level operations like detectCollision() to callers.
Composition	Instead of inheriting from HandleCollision or DetectCollision, the manager stores them as private fields and uses them to handle specific tasks. This keeps the components independent and lets the manager coordinate their work without tight coupling.

3.3.2 TimeManager

Principle	**How It Is Applied in TimeManager**
Encapsulation	All time-related state (deltaTime, totalTime, timeScale, isPaused) is private. The time itself cannot be directly manipulated by external systems; instead, the public API must be used, which enforces proper usage, such as clamping timeScale.
Single Responsibility	TimeManager has only one concern: time management. TimeManager does not deal with rendering, input, collision, or any other aspect of the game logic, only time management and time flow control.
Singleton Pattern	TimeManager is a Singleton class, using getInstance(), to ensure there is only one time source in the entire application. This avoids any potential time state conflicts.
Abstraction	The internal processing (delta time capping, time scale clamping, pause handling) is not visible to the caller. The caller can only see the final processed deltaTime value, not the processing that occurred.

3.3.3 Entity Manager

OOP Principle	How it is Applied in Scene Manager
Abstraction	Entity Manager provides (allows) high level interface for the game engine to interact (engage) with game objects so that it does not need to understand the internal complexities of the entity list management or the "pending changes" logic. It abstracts the specific implementation of how an entity updates or renders by calling generic update() and render() methods.
Encapsulation	Data integrity is maintained by making the entity lists (entities, entitiesToAdd, entitiesToRemove) private. External classes are unable to modify these lists directly, hence they must use public methods such as addEntity() or removeEntity(). Furthermore, getEntities() also returns a shallow copy of the list to prevent external code from modifying the manager's internal state.
Polymorphism	Overloading:The manager uses method overloading with update() and updateAll(float deltaTime), allowing the engine to trigger updates with or without a specific time step. Dynamic Binding(Overriding):When the manager calls entity.update(deltaTime), it performs a polymorphic call. Even though the manager holds a list of the generic Entity type, Java will still execute the specific overridden update logic of the actual subclass (e.g., a Player or Enemy subclass).

Inheritence	The manager relies on the IS-A relationship where various game objects (such as players or enemies) are inherited from the base Entity class. This allows the Entity Manager to store and process diverse objects within a single List<Entity>.
Composition	The Entity Manager demonstrates composition as it HAS-A collection of Entity objects. It is responsible for their lifecycle, when the manager is disposed of via the dispose() method, it explicitly calls entity.Dispose() on all contained entities, ensuring proper resource management.
Interfaces/Abstract Classes	class is designed to work with the Entity type, which functions as an abstract class or interface. By using the generic method <T extends Entity> List<T> getEntitiesByType(Class<T> type), the manager can dynamically handle any subtype defined in the system.

3.3.4 SceneManager

Principle	**How It Is Applied in SceneManager**
Abstraction	SceneManager stores all scenes through the abstract Scene type. It has no knowledge of — and no dependency on — MenuScene, MainScene, or EndScene. Any new scene type added to the engine in the future will be automatically compatible with SceneManager without any changes to the manager itself.
Encapsulation	Both attributes (scenes and currentScene) are declared private. External classes cannot directly manipulate the scene map or change the active scene pointer; they must use the public methods, which enforce the correct lifecycle ordering (stop-then-start) on every transition.
Polymorphism	When SceneManager calls currentScene.update(dt) or currentScene.render(), Java's runtime dispatch automatically invokes the correct overridden implementation in the concrete subclass (MenuScene, MainScene, or EndScene). The manager needs only one line of code to drive any scene, regardless of how complex that scene's internal logic is.
State Pattern	The SceneManager is a direct implementation of the State design pattern: the engine's overall behaviour changes depending on which scene (state) is current. Each state fully encapsulates its own update and render logic, and transitions are mediated solely through SceneManager rather than scattered across the codebase.

3.3.5 Input/Output Manager

OOP Principle	How it is Applied in Scene Manager
Abstraction	The manager interacts with only the abstract interfaces keyboard, mouse, and display. It does not depend on any specific implementation, allowing the engine to switch between simulation devices and libgdx-backed devices
Encapsulation	All internal state is kept private. External systems cannot modify raw input states directly.
Polymorphism	Multiple concrete classes implement the same interfaces (keyboard, mouse display) the IO Manager itself can then work with any implementation at runtime. Enabling seamless switching between simulation mode and real hardware input through LibGDX

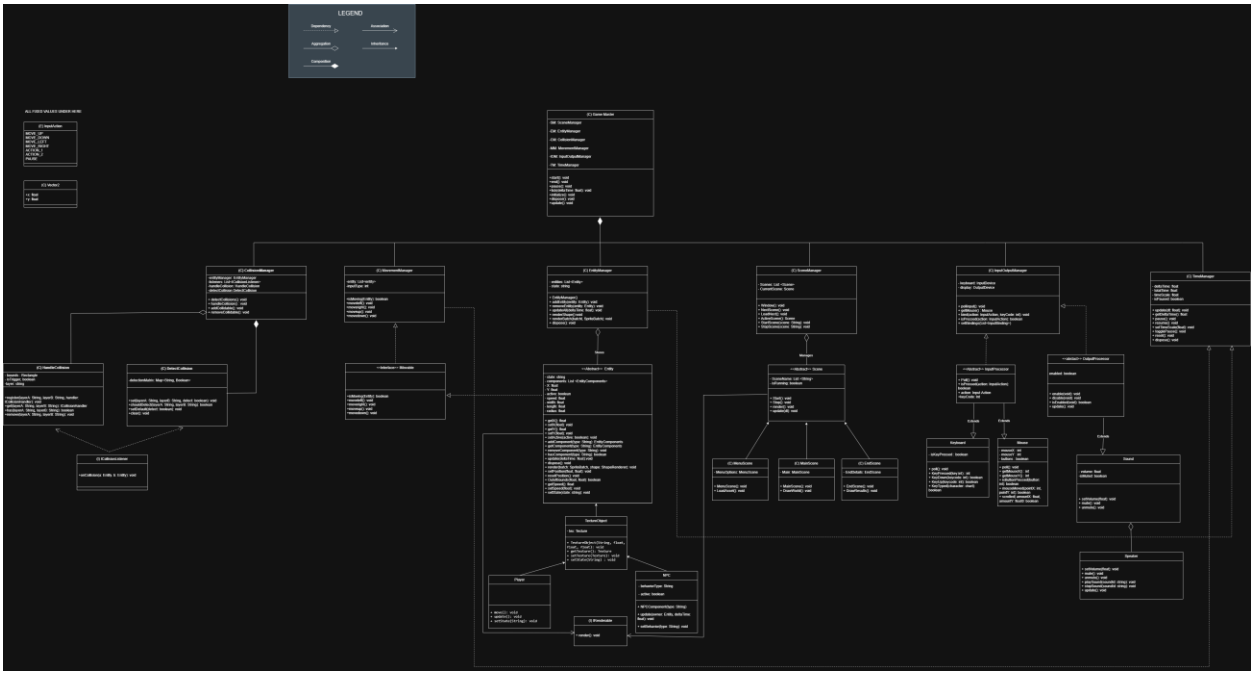
Dependency Inversion	High-level engine systems depend on abstractions, not LibGDX directly. This keeps the engine flexible, testable, and free from context-specific logic.
Single Responsibility	The manager has one clear responsibility, to handle user input and output.

3.3.6 MovementManager

Principle	How It is Applied in MovementManager
Abstraction	IMovable interface defines movement contract without exposing implementation. Entities implement movement through abstract update() methods, hiding behavior logic from external systems
Encapsulation	Position, velocity, and speed attributes are protected/private. External code must use public methods (setPosition, move) to modify movement state, preventing invalid modifications
Inheritance (Is-A)	Player and NPC both inherit from Entity base class, inheriting common movement attributes (position, velocity, speed). Specialized movement logic is added through overridden update() methods
Polymorphism	Entity Manager calls entity.update(deltaTime) polymorphically—runtime type determines whether Player.update() or NPC.update() executes. Same interface handles different movement behaviors
Composition (Has-A)	Entity HAS-A Vector2 for position and velocity rather than extending Vector2. This composition allows flexible position management

3. UML Diagram

OBJ



link: https://app.diagrams.net/#G1MP2k38Algj7eYTQHOzTlL6hGhX5lZ-x7#%7B%22pageId%22%3A%22hK1sMXyNA6UtTFOqNo5U%22%7D

4. Team Contribution

Student	Contributions Made
Abburu Sai Tanvi	Created the initial code skeleton that defined the structure of the Abstract Engine, laying the groundwork for the team's implementation. Made significant contributions to core system coding, integrating the various managers to ensure appropriate and consistent interactions during execution. Fully implemented the CollisionManager, including collision detection and entity interactions for the UML, report and slides. Worked with the AI teammate during the ideation phase to develop and refine the sixth manager, ensuring it fit smoothly within the overall architecture. Maintained consistency between the implementation and UML design, making adjustments as needed, and initiated the structure of the project slides and report.
Chourasia Anushka	Implemented the TimeManager, offering a centralized time management service and frame-independent updates for all engine systems. Implemented the entire time management functionality (delta time, time scaling, pause/resume) using the Singleton design pattern. Integrated with GameMaster and synchronized with EntityManager and MovementManager for proper timing. Wrote all the TimeManager-related documentation, including the technical report section, UML diagrams, implementation code with comments, and presentation (slides and scripts).

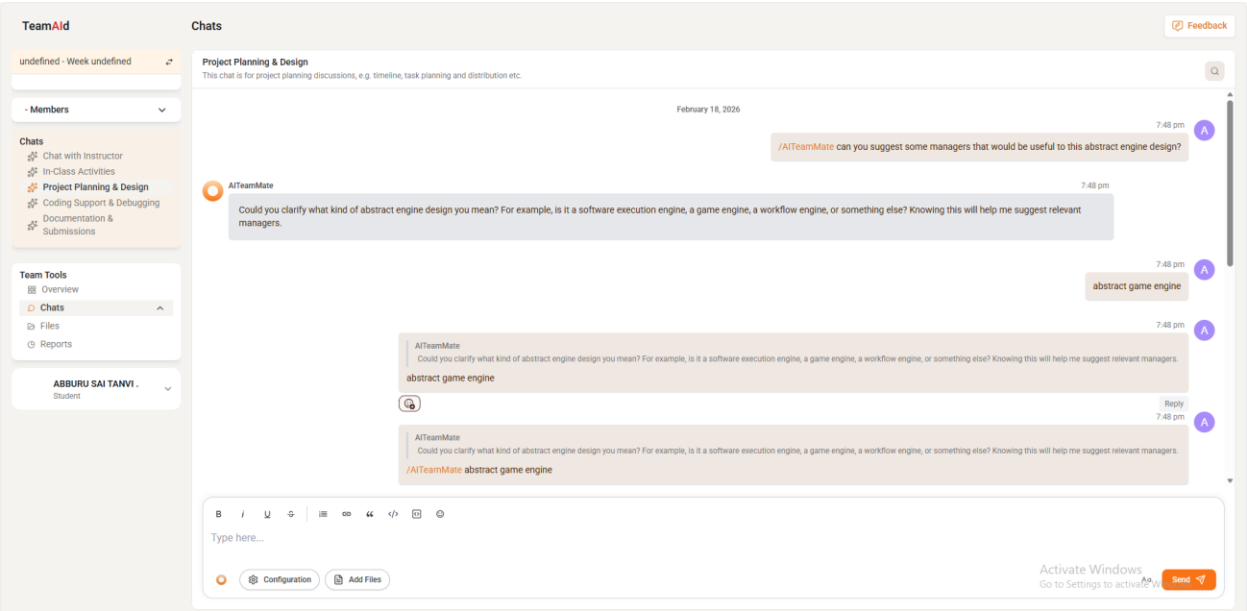
	Adhered to OOP guidelines and consistency between code, UML, and documentation throughout the implementation.
Chern De Qian	Developed the foundational code skeleton for the Abstract Engine, establishing the core structure that guided the team's subsequent implementation. Designed and implemented the SceneManager and complete scene system. Applied State Pattern for game flow control, implemented dual transition mechanisms, and integrated with all managers. Responsible for scene lifecycle design, UML class structure, and ensuring proper resource management during transitions. Led development of the abstract game engine's architecture, focusing on scene management and state control to coordinate all six manager systems. Ensured alignment between the implementation and the UML design, updating the model where necessary to reflect architectural changes.
Ang Wei Long	Responsible for entity manager that handle the full lifecycle (creation, update, disposal) of all game objects. To achieve system stability, I developed a buffer-based logic for entity additions and removals to prevent any concurrent modification errors during the game loop. I also used six OOP principles (Abstraction, Encapsulation, Inheritance, Composition, Abstract Classes, and Polymorphism) within the entity system to ensure scalability. Type specific filtering (getEntitiesByType) and resource management (dispose) were also implemented for efficient engine operations. Edited the final presentation video, ensuring that the team demonstration was well-paced, clear, and met the 10-minute strict duration requirement.
Adora Goh Shao Qi	Responsible for the design and documentation of the Input/Output Manager, including defining its architecture, attributes, and key methods. Contributed to the UML structure by outlining the IO components and ensuring alignment with the overall engine design. Prepared the IO Manager section of the report and presentation slides, and assisted in refining the clarity, structure, and consistency of the team's documentation
Goh Jason Fadil	Major Contributions: Heavily involved in the Movement Manager Component (UML/Report/PPT). Involved in paving the general direction of the project to the ideation of our Abstract Engine. Minor Contributions: Fixed up gaps, edited and changed parts of the UML diagram throughout all managers.
TeamAId	The AI team member aided in the sixth manager's ideation phase for the team, providing ideas on designs and how to incorporate it with the Abstract Engine. These ideas helped the team explore various options, which were then refined and implemented to work well with the entire system.

5. Appendix A: Program Structure

```
AbstractEngine/
├── src/
│   ├── engine/
│   │   ├── core/
│   │   │   └── GameMaster.java          # Main engine controller
│   │   ├── managers/
│   │   │   ├── SceneManager.java        # Scene lifecycle management
│   │   │   ├── EntityManager.java       # Entity lifecycle management
│   │   │   ├── CollisionManager.java    # Collision detection & response
│   │   │   ├── MovementManager.java     # Movement system
│   │   │   ├── InputOutputManager.java  # Input/output handling
│   │   │   └── TimeManager.java         # Time management & delta time
│   │   ├── scene/
│   │   │   ├── Scene.java               # Abstract scene base class
│   │   │   ├── MenuScene.java           # Menu scene implementation
│   │   │   ├── MainScene.java           # Main simulation scene
│   │   │   └── EndScene.java            # End/results scene
│   │   ├── entities/
│   │   │   ├── Entity.java              # Abstract entity base class
│   │   │   ├── Player.java              # Player entity
│   │   │   ├── NPC.java                 # Non-player character
│   │   │   ├── TextureObject.java       # Visual simulation object
│   │   │   └── IMovable.java            # Movable interface
│   │   ├── collision/
│   │   │   ├── ICollisionListener.java  # Collision event interface
│   │   │   ├── DetectCollision.java     # Collision detection logic
│   │   │   └── HandleCollision.java     # Collision response handling
│   │   ├── input/
│   │   │   ├── InputAction.java         # Input action enumeration
│   │   │   ├── InputProcessor.java      # Input processor interface
│   │   │   ├── Keyboard.java            # Keyboard input handler
│   │   │   └── Mouse.java               # Mouse input handler
│   │   ├── output/
│   │   │   ├── OutputProcessor.java     # Abstract output processor
│   │   │   ├── SoundOut.java            # Sound output interface
│   │   │   ├── Sound.java               # Sound resource class
│   │   │   └── Speaker.java             # Audio output device
│   │   └── utils/
│   │       └── Vector2.java             # 2D vector mathematics
│   └── simulation/
│       ├── EngineSimulation.java        # Demo simulation
│       └── PlayableGame.java            # Complete playable simulation demo
```

6. Appendix B: TeamAId Usage

Appendix B shows some screenshots of some questions we asked AI – Teammate chat bot. These screenshots are proof of our use with TeamAId



TeamAId

Chats

Feedback

undefined - Week undefined

- Members

Chats
- Chat with Instructor
- In-Class Activities
- Project Planning & Design
- Coding Support & Debugging
- Documentation & Submissions

Team Tools
- Overview
- Chats
- Files
- Reports

ABBURU SAI TANVI .
Student

Project Planning & Design
This chat is for project planning discussions, e.g. timeline, task planning and distribution etc.

AITeamMate 7:48 pm

> AITeamMate
> Could you clarify what kind of abstract engine design you mean? For example, is it a software execution engine, a game engine, a workflow engine, or something else? Knowing this will help me suggest relevant managers.

For an abstract game engine, typical managers you might consider include:
- Entity Manager (handles entities and their components)
- Resource Manager (manages game assets like textures, sounds, models)
- Input Manager (handles user inputs from keyboard, mouse, controller)
- Scene Manager (manages different game scenes or levels)
- Audio Manager (manages sound effects and music playback)
- Physics Manager (handles physics simulations and collisions)
- AI Manager (coordinates NPC behaviors)
- Render Manager (manages rendering pipeline and graphics)
- Event Manager (handles game events and messaging between systems)

Would you want recommendations tailored to any particular game genre or platform?

7:49 pm
/AITeamMate how about a Time Manager?

AITeamMate 7:49 pm

A Time Manager in an abstract game engine is a solid addition. It typically handles:
- Game time tracking (elapsed time, delta time between frames)
- Time scaling (slow motion, speed up effects)
- Scheduling timed events or callbacks

Type here...

Configuration | Add Files | Send

Activate Windows
Go to Settings to activate Windows.

Click to go back (Alt+Left arrow), hold to see history

TeamAId

Chats

Feedback

undefined - Week undefined

- Members

Chats
- Chat with Instructor
- In-Class Activities
- Project Planning & Design
- Coding Support & Debugging
- Documentation & Submissions

Team Tools
- Overview
- Chats
- Files
- Reports

ABBURU SAI TANVI .
Student

Project Planning & Design
This chat is for project planning discussions, e.g. timeline, task planning and distribution etc.

- Resource Manager (manages game assets like textures, sounds, models)
- Input Manager (handles user inputs from keyboard, mouse, controller)
- Scene Manager (manages different game scenes or levels)
- Audio Manager (manages sound effects and music playback)
- Physics Manager (handles physics simulations and collisions)
- AI Manager (coordinates NPC behaviors)
- Render Manager (manages rendering pipeline and graphics)
- Event Manager (handles game events and messaging between systems)

Would you want recommendations tailored to any particular game genre or platform?

7:49 pm
/AITeamMate how about a Time Manager?

AITeamMate 7:49 pm

A Time Manager in an abstract game engine is a solid addition. It typically handles:
- Game time tracking (elapsed time, delta time between frames)
- Time scaling (slow motion, speed up effects)
- Scheduling timed events or callbacks
- Managing global timers or countdowns
- Synchronizing animations, physics steps, and gameplay updates with time

It helps keep all time-dependent systems consistent and decoupled from real-world clock variations. Would you want it designed as a standalone manager or integrated with other systems like physics or animation?

Type here...

Configuration | Add Files | Send

Activate Windows
Go to Settings to activate Windows.